Work Trail — AI Build Plan

Build objective

Build an MVP called **Work Trail**: a responsive admin web dashboard and Android-first React Native employee app. Do not implement features outside the MVP until the acceptance criteria below pass.

Architecture

```
Admin web (Next.js) ─┐
                     ├── API (NestJS or FastAPI) ── PostgreSQL + PostGIS
Employee app (Expo) ─┘                ├── Object storage for PDFs
                                      ├── Queue/worker for AI and reports
                                      ├── Maps/reverse geocoding provider
                                      └── LLM provider
```

Recommended implementation choices

- **Admin dashboard:** Next.js, TypeScript, Tailwind CSS, shadcn/ui, TanStack Query, React Hook Form, Zod.
- **Employee app:** React Native with Expo development build, TypeScript, Expo Router, TanStack Query, Zustand, secure token storage, local SQLite queue.
- **Location:** Expo Location plus TaskManager/background task and an Android foreground service-compatible setup. Use a development build, not only Expo Go, for realistic testing.
- **Backend:** NestJS with TypeScript, Prisma, PostgreSQL with PostGIS. A FastAPI equivalent is acceptable if it preserves the same contracts.
- **Auth:** JWT access/refresh tokens, bcrypt/argon2 password hashing, roles enforced on API.
- **Maps:** Google Maps platform or Mapbox; use one provider consistently.
- **Jobs:** Redis + BullMQ for AI summaries and PDF generation.
- **PDF:** HTML template rendered through Puppeteer/Playwright on server.

Repository layout

```
work-trail/
  apps/
    admin-web/
    employee-mobile/
    api/
  packages/
    types/
    api-client/
    ui/
```

```
docs/
  prd.md
  plan.md
  api-contract.md
  data-model.md
  ai-prompts.md
  build-prompt.md
infra/
  docker-compose.yml
  migrations/
README.md
```

Use a monorepo (pnpm workspaces or Turborepo). Share TypeScript request/response types between clients and API.

Delivery phases

Phase 0 — Foundation

1. Create monorepo, formatting, linting, environment variable validation, Docker Compose for PostgreSQL/Redis.
2. Add database migrations and seed a demo company/admin/employees.
3. Implement API health endpoint, OpenAPI docs, error envelope, request logging, and rate limits.
4. Build a basic design system: Work Trail wordmark, navigation shell, cards, tabs, charts placeholders, empty/loading/error states.

Done when: a new developer can run the stack locally with one command and sign in with seeded credentials.

Phase 1 — Auth and organisation isolation

1. Implement companies, users, employees, invitations, refresh tokens, password reset.
2. Build admin signup/login/logout/reset screens.
3. Build employee login and invitation registration screens.
4. Add role guards and organisation-scoped database queries.

Done when: an admin from Company A cannot read Company B data even by manipulating IDs.

Phase 2 — Employee work session

1. Build employee Home screen and permissions disclosure screen.
2. Implement `Start work`, active timer, `End work`, session restore, and local state.
3. Add immediate start/end location capture API calls.
4. Display last sync time and clear tracking state.

Done when: starting and ending a session creates accurate server records and prevents a second concurrent session.

Phase 3 — Location pipeline

1. Configure Android permissions and foreground notification.
2. Register a background location task to target a five-minute interval while a session is active.
3. Build SQLite outbox with retry/backoff/idempotency key.
4. Build batch upload endpoint and validate coordinates, timestamps, accuracy, and session ownership.
5. Add a physical-device test checklist.

Done when: location points survive temporary offline connectivity and appear in the database without duplicates.

Phase 4 — Employee updates and speech input

1. Build Add Update screen and text validation.
2. Add microphone permission and speech-to-text integration; write transcript into editable textarea.
3. Implement update submit and offline queue fallback.
4. Add update history.

Done when: employee can type or speak an update, edit it, submit it, and see its processing state.

Phase 5 — Admin dashboard and employee detail

1. Build KPI dashboard using real aggregate endpoints.
2. Build employee list/search/status filters.
3. Build employee detail page with tabs.
4. Build map view with range selector, polyline, start/end markers, data-gap indication, and location-point inspection.
5. Build updates timeline and work-hours table.

Done when: admin can choose an employee and inspect today/week/month route, updates, and total hours.

Phase 6 — AI reports and PDF

1. Implement a queue worker for single-update summaries and daily/weekly/monthly aggregated summaries.
2. Use strict JSON-schema output; validate and retry malformed LLM responses.
3. Build report preview and PDF endpoint.

4. Render secure PDF and store file metadata; enforce admin-only access.

Done when: admin downloads an employee PDF containing selected-period hours, summaries, and updates.

Phase 7 — Hardening and pilot

1. Add audit events, monitoring, backup policy, error tracking, and basic admin support tools.
2. Add privacy copy, consent record, delete/export workflow, data-retention configuration.
3. Test on several Android brands, low battery, battery saver, screen locked, no network, restored network, and app restart.
4. Run a five-company pilot before broad launch.

Build order rule

Do not build the AI, reporting, or polished map analytics before the full path works: invite -> employee login -> start work -> sync location -> submit update -> end work -> admin sees data.

Definition of done

- TypeScript compiles with no errors.
- Lint and unit tests pass.
- Every endpoint has authentication and organisation authorisation tests.
- All UI states have loading, empty, success, and failure handling.
- No API key or secret is committed to the repository.
- Physical Android tracking test is documented with observed OS behaviour.